

RELATÓRIO DE TESTES

Motor de Cálculos — TechStore E-commerce

Autor:

Vitor Razia Bubols

Disciplina:

Desenvolvimento de Software

Professor:

Rafael Coelho Ventura Ferramenta

Modalidade:

Individual

Data de entrega:

29 de abril de 2026

1. Plano de Testes

1.1 Objetivo do Plano

Este plano de testes tem como objetivo validar a implementação do motor de cálculos da TechStore, composto pelos módulos `coreMath.js` e `businessLogic.js`, por meio de testes automatizados com Vitest. A meta é garantir que todas as operações matemáticas básicas e as regras de negócio derivadas se comportam corretamente tanto em cenários válidos quanto em entradas inválidas, assegurando que nenhuma funcionalidade entre em produção sem cobertura adequada de testes.

1.2 Escopo

Dentro do Escopo	Fora do Escopo
Funções <code>add</code> , <code>subtract</code> , <code>multiply</code> , <code>divide</code> (<code>coreMath.js</code>)	Testes de interface ou camada HTTP
Funções <code>calculateAverage</code> e <code>calculateDiscountedPrice</code> (<code>businessLogic.js</code>)	Testes de performance ou carga
Casos válidos, inválidos, limites e erros de cada função	Testes end-to-end de fluxo de compra
Verificação da restrição: <code>businessLogic.js</code> não usa operadores nativos	Testes de banco de dados ou persistência

1.3 Ferramentas Utilizadas

Ferramenta	Versão	Finalidade
Vitest	^4.0.18	Framework principal de testes unitários e de integração
Node.js	18+	Ambiente de execução JavaScript
npm	9+	Gerenciamento de dependências do projeto
VS Code	Qualquer	Editor de código utilizado no desenvolvimento

1.4 Estratégia de Testes

Os testes foram organizados em duas fases que refletem diretamente a arquitetura do projeto. A Fase 1 cobre o módulo `coreMath.js` com testes puramente unitários: cada função (`add`, `subtract`, `multiply`, `divide`) é testada de forma completamente isolada, sem nenhuma dependência externa. Isso garante que a fundação matemática do sistema está correta antes de qualquer outra coisa.

A Fase 2 cobre o módulo `businessLogic.js` com testes de integração: as funções `calculateAverage` e `calculateDiscountedPrice` dependem diretamente das funções de `coreMath.js`, o que significa que um erro no módulo base pode propagar falhas para o nível de negócio. Essa separação é intencional e pedagogicamente importante — ela demonstra como testes unitários e de integração se complementam numa arquitetura em camadas.

Em ambas as fases, os testes seguem o padrão AAA (Arrange, Act, Assert) e cobrem três categorias de cenário: happy path (entradas válidas com resultado esperado), casos de erro

(entradas inválidas que devem lançar exceções via `toThrow()`), e boundary values (como desconto de 0% e 100%, ou array de um único elemento). A restrição de que `businessLogic.js` não pode usar operadores nativos (+, -, *, /) foi verificada manualmente durante a implementação.

1.5 Critérios de Aceite

Critério	Condição de Aceite
Todos os testes passando	<code>npx vitest</code> exibe 0 failed
Cobertura mínima <code>coreMath</code>	<code>coreMath.test.js</code> com mínimo de 12 testes
Cobertura mínima <code>businessLogic</code>	<code>businessLogic.test.js</code> com mínimo de 8 testes
Restrição de operadores nativos	<code>businessLogic.js</code> usa apenas funções de <code>coreMath.js</code> para aritmética
Cobertura de casos de erro	Todos os cenários de lançamento de erro cobertos com <code>toThrow()</code>

2. Matriz de Rastreabilidade de Testes

A tabela abaixo mapeia cada caso de teste ao comportamento específico que ele valida. Os IDs seguem o padrão CM (coreMath) e BL (businessLogic). Testes do módulo coreMath são classificados como Unitários; testes do businessLogic são classificados como Integração, pois dependem das funções do módulo base.

ID	Módulo	Função	Cenário Testado	Dados de Entrada	Resultado Esperado	Tipo
CM-01	coreMath	add()	Soma de dois positivos	add(2, 3)	5	Unitário
CM-02	coreMath	add()	Soma com valor negativo	add(5, -3)	2	Unitário
CM-03	coreMath	add()	Soma com zero	add(7, 0)	7	Unitário
CM-04	coreMath	subtract()	Subtração básica	subtract(10, 4)	6	Unitário
CM-05	coreMath	subtract()	Resultado negativo	subtract(3, 8)	-5	Unitário
CM-06	coreMath	subtract()	Subtração com zero	subtract(5, 0)	5	Unitário
CM-07	coreMath	multiply()	Multiplicação de dois positivos	multiply(4, 3)	12	Unitário
CM-08	coreMath	multiply()	Multiplicação por zero	multiply(9, 0)	0	Unitário
CM-09	coreMath	multiply()	Resultado negativo	multiply(5, -2)	-10	Unitário
CM-10	coreMath	divide()	Divisão exata	divide(10, 2)	5	Unitário
CM-11	coreMath	divide()	Resultado decimal	divide(7, 2)	3.5	Unitário
CM-12	coreMath	divide()	Divisão por zero	divide(5, 0)	Lança erro	Unitário
BL-01	businessLogic	calculateAverage()	Média de array com múltiplos valores	[10, 20, 30]	20	Integração
BL-02	businessLogic	calculateAverage()	Array com único elemento	[42]	42	Integração
BL-03	businessLogic	calculateAverage()	Array vazio	[]	Lança erro	Integração
BL-04	businessLogic	calculateAverage()	Entrada inválida (null)	null	Lança erro	Integração

ID	Módulo	Função	Cenário Testado	Dados de Entrada	Resultado Esperado	Tipo
BL-05	businessLogic	calculateAverage()	Entrada inválida (string)	"texto"	Lança erro	Integração
BL-06	businessLogic	calculateDiscountedPrice()	Desconto válido 10%	(200, 10)	180	Integração
BL-07	businessLogic	calculateDiscountedPrice()	Desconto 0% — preço original	(150, 0)	150	Integração
BL-08	businessLogic	calculateDiscountedPrice()	Desconto 100% — retorna zero	(100, 100)	0	Integração
BL-09	businessLogic	calculateDiscountedPrice()	Desconto maior que 100%	(100, 110)	Lança erro	Integração
BL-10	businessLogic	calculateDiscountedPrice()	Preço zero ou negativo	(0, 10)	Lança erro	Integração

Total: 22 casos de teste — 12 unitários (coreMath) e 10 de integração (businessLogic), superando o mínimo exigido de 20 testes.

2.1 Print do terminal:

```
PS C:\Users\vitor\Git\SENAI_FASE3\Teste_de_sistemas\TESTES_UNITARIOS\e-commerce\02-calc-project> npx vitest
DEV v1.6.1 C:/Users/vitor/Git/SENAI_FASE3/Teste_de_sistemas/TESTES_UNITARIOS/e-commerce/02-calc-project
✓ tests/coreMath.test.js (12)
✓ tests/businessLogic.test.js (10)

Test Files  2 passed (2)
Tests      22 passed (22)
Start at   01:57:09
Duration   491ms (transform 43ms, setup 0ms, collect 73ms, tests 11ms, environment 0ms, prepare 376ms)

PASS Waiting for file changes...
press h to show help, press q to quit
```

3. Reflexão Escrita

3.1 Qual foi o teste mais difícil de escrever?

O teste mais desafiador foi o BL-02, que valida calculateAverage com um array de um único elemento. À primeira vista parece trivial — a média de um array com um número é o próprio número. Mas o ponto de atenção foi garantir que a implementação realmente usa divide() de coreMath.js para calcular a média (dividindo a soma por 1), e não simplesmente retorna o primeiro elemento diretamente. Escrever o teste foi simples; o desafio foi entender o que

estava sendo testado de verdade: o comportamento correto do algoritmo, não só o resultado final.

3.2 O que a restrição de não usar operadores nativos ensinou?

A regra de que `businessLogic.js` não pode usar `+`, `-`, `*`, `/` diretamente foi a parte mais interessante do projeto. No início pareceu uma limitação artificial, mas na prática ela forçou uma reflexão importante sobre composição de módulos: ao usar `add()`, `subtract()`, `multiply()` e `divide()` como blocos de construção, qualquer bug no cálculo matemático pode ser rastreado até o módulo exato onde está — e os testes de cada camada isolam o problema rapidamente.

Isso também evidenciou a diferença entre testes unitários e de integração na prática. Se um teste de `businessLogic` falha, a primeira pergunta é: o problema está na lógica de negócio ou na função matemática que ela chama? Com os testes de `coreMath` todos passando de forma independente, a resposta fica muito mais clara.

3.3 Como foram decididos os cenários de teste?

A decisão seguiu a mesma lógica de projetos anteriores, mas com uma camada extra de complexidade na Fase 2. Para `coreMath`, os critérios foram: um caso feliz básico, um caso com número negativo e um com zero — cobrindo as três situações mais comuns de entrada. Para `divide()`, o caso de divisão por zero é obrigatório por ser uma regra crítica explicitamente documentada.

Para `businessLogic`, além dos happy paths e erros esperados, foi preciso pensar nos casos de borda específicos do domínio: array com um elemento (que testa o caminho mínimo do algoritmo de média), desconto de 0% (que deve retornar o preço intacto sem erros) e desconto de 100% (que deve retornar zero, não negativo). Esses valores-limite são os mais propensos a revelar falhas de lógica condicional.

3.4 Qual a diferença prática entre testes unitários e de integração neste projeto?

Neste projeto a distinção ficou muito concreta. Os testes de `coreMath` são unitários no sentido mais puro: cada função é testada de forma totalmente isolada, sem importar ou depender de nenhum outro módulo do projeto. Se `add(2, 3)` retornar 6, o teste falha — simples assim.

Já os testes de `businessLogic` são de integração porque as funções testadas dependem diretamente de `coreMath.js`. Quando `calculateDiscountedPrice(200, 10)` é chamada, ela internamente usa `multiply()` e `divide()` para calcular o desconto e `subtract()` para aplicá-lo. O teste verifica não só que o resultado final está correto, mas implicitamente que a integração entre os dois módulos funciona. É por isso que a ordem importa: os testes unitários precisam passar primeiro para que os de integração façam sentido.

3.5 Considerações Finais

Este projeto foi uma evolução natural em relação ao anterior. Sair de funções completamente independentes para um sistema em duas camadas trouxe complexidade real: agora um bug em `coreMath` pode quebrar testes em `businessLogic`, e saber identificar onde está o problema é uma habilidade que só se desenvolve com prática. Trabalhar com a restrição dos operadores nativos também foi valioso — não como limitação, mas como exercício de design.

de software. A atividade reforçou que testes não são apenas uma rede de segurança, mas uma forma de documentar decisões de arquitetura e deixar explícito o contrato de cada função do sistema.